

Minimal Repro: Multi-line LaTeX macro + footnote → Extra }

Tests whether parse-latex expands user-defined macros during PDF builds, and whether pandoc itself expands them independently

2026-04-14

Context

In a Quarto book (HTML + PDF), every chapter includes `latex/latex-commands.qmd` via `{{<include >}}`. That file defines fallback index macros for HTML contexts:

```
\providecommand{\ixd}[1]{%  
  \index{#1@\texttt{#1} data}%  
  \index{datasets!#1@\texttt{#1}}%  
}
```

Standard LaTeX multi-line style: % at line-ends suppresses whitespace in the macro *expansion* (XeLaTeX interprets % as a comment, discards the newline). This is correct and normal for LaTeX. When XeLaTeX expands `\ixd{Prestige}`, it produces `\index{...}\index{...}` with no % in the result.

The problem: Something during the *pandoc pass* expands `\ixd{Prestige}` *literally*, inserting the % characters from the definition body into the output text. When a Markdown footnote (`. [^fn]`) follows in the source, the output becomes:

```
\index{Prestige@\texttt{Prestige} data}%  
\index{datasets!Prestige@\texttt{Prestige}}%.\footnote{...}
```

The `%. \footnote{` means XeLaTeX never opens the footnote → ! Extra }, or forgotten `\endgroup`.

Setup: `parse-latex` is in format: `html: filters: only` (not `global filters:`). Per @mcanouil, `FORMAT = "latex"` during the PDF pandoc pass, so the filter's `if not`

FORMAT:match 'html' then return {} end guard should fire and do nothing. **If that is correct, the expansion must come from a different mechanism.**

Test 1: Static macro + footnote (parse-latex in HTML filters only)

A raw LaTeX block defines a multi-line macro; a raw inline uses it; a Markdown footnote follows immediately. This is the minimal reproduction of the book scenario.

Static inline then footnote: *T1: alpha*.¹

Test 2: Definition via `{{< include >}}`

In the actual book, the `\providecommand` comes from an included file, not directly in the chapter source. Quarto resolves `{{< include >}}` before the pandoc pass, so the included content ends up in the freeze cache (`tex.json`). Any stale cache will embed the *old* multi-line definition even after the source file is updated.

T2: beta.²

Test 3: Inline R output near footnote

The book uses inline R expressions that emit raw LaTeX macros for PDF:

```
dataset("Prestige") → "\texttt{Prestige}\idx{Prestige}" (for PDF)
```

¹Test 1 footnote. **If PDF build fails here** with `Extra }`, something expanded `\myone` during the pandoc pass (inserting literal `%`). Check the generated `.tex` file for: - PASS: `\myone{alpha}.\footnote{Test 1 footnote...}` (unexpanded, one line) - FAIL: `\textit{T1: alpha}%\n.\footnote{...}` (expanded, `%` before footnote)

²Test 2 footnote. If this fails but Test 1 passes, the issue is specific to `{{< include >}}`'d definitions (e.g., a stale freeze cache embedding the definition from a previous build).

This test simulates that without the full book infrastructure:

T3: gamma

³
.

How to run

```
# From the project root:
quarto render issues/quarto-pandoc-test.qmd --to pdf
quarto render issues/quarto-pandoc-test.qmd --to html
```

To test WITHOUT parse-latex (isolates pandoc's own behavior): Remove **filters:** [tarleb/parse-latex] from the YAML and rebuild. If the error persists, pandoc itself is expanding the macros — parse-latex is not involved.

To test WITHOUT {{< include >}}: Remove the **{{< include >}}** line and \mytwo/\mythree tests.

To diagnose which mechanism is expanding macros, add this Lua filter alongside parse-latex (per @mcanouil's suggestion) to confirm FORMAT during the PDF pass:

```
-- debug-format.lua
function Pandoc(doc)
  io.stderr:write("[debug] FORMAT = " .. tostring(FORMAT) .. "\n")
  return doc
end
```

Expected .tex content (with keep-tex: true)

If macros are **NOT** expanded during pandoc pass (correct):

³Test 3 footnote. If this fails but Test 1 passes, the issue is specific to knitr-emitted raw LaTeX content and how Quarto's freeze cache stores it. (\mythree is defined in quarto-pandoc-test-macros.qmd, included in Test 2.)

```
\myone{alpha}.\footnote{Test 1 footnote...}  
\mytwo{beta}.\footnote{Test 2 footnote...}
```

XeLaTeX then expands these natively; the % in the definition body acts as a TeX comment (discards the newline), so the expansion contains no %.

If macros ARE expanded during pandoc pass (the bug):

```
\textit{T1: alpha}%  
.\footnote{Test 1 footnote...}
```

The literal % from the definition body is inserted into the pandoc output, commenting out the .\footnote{ opener.